

Working with resources

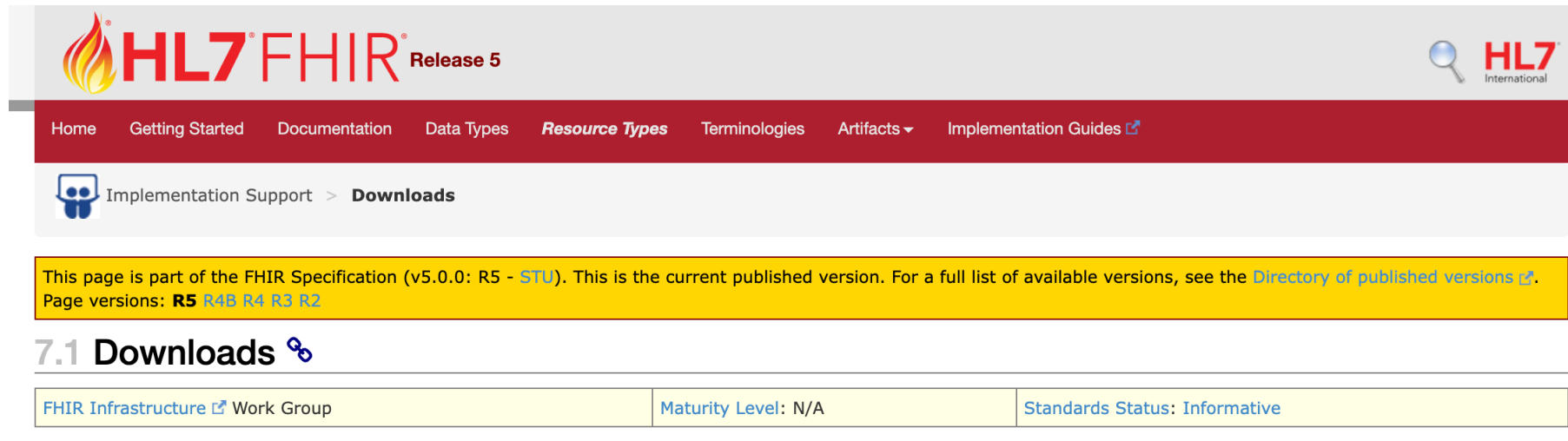
eHealth Master SS2023

Sten Hanke

CREATE PATIENT RESOURCE

- Installed VSCode
- Installed Insomnia or Postman

- HL7.org/FHIR
- Documentation / Downloads → Download JSON schemas



The screenshot shows the HL7 FHIR Release 5 website. The header includes the HL7 FHIR logo and navigation links: Home, Getting Started, Documentation, Data Types, Resource Types, Terminologies, Artifacts, and Implementation Guides. Below the header, there is a breadcrumb trail: Implementation Support > Downloads. A yellow banner contains a disclaimer: "This page is part of the FHIR Specification (v5.0.0: R5 - STU). This is the current published version. For a full list of available versions, see the Directory of published versions." Below the banner, the section title "7.1 Downloads" is displayed. At the bottom, a table provides details about the FHIR Infrastructure Work Group, its maturity level, and its standards status.

FHIR Infrastructure Work Group	Maturity Level: N/A	Standards Status: Informative
--------------------------------	---------------------	-------------------------------

- Make a new folder for your patient resource
- Patient resource (everything beside the Name can be faked):
 - *Name (Vor- und Familienname)*
 - *Geschlecht*
 - *Geburtsdatum*
 - *Telefon*
 - *Email*
 - *Vollständige Adresse inkl. Postleitzahl*

- Open folder with VSCode
- Copy fhir.schema.json in folder
 - The file contains ALL FHIR resources

- Open Command Palette
 - Mac OS: command+shift+P
- Open → Preferences: Open Workspace Settings (JSON)
- This is a JSON by itself with many auto complete functions
- Autocomplete json.schema
- Configure it for FHIR files. Should look like this:
- Create a new file patient.fhir.json



The screenshot shows the VS Code interface with two tabs: 'settings.json' and 'patient.fhir.json'. The 'settings.json' tab is active, displaying the following JSON configuration:

```
.vscode > {} settings.json > ...
1  {
2    "json.schemas": [{
3      "fileMatch": [
4        "*.fhir.json"
5      ],
6      "url": "./fhir.schema.json"
7    }]
8  }
```

- Go to the FHIR patient resource (HL7.org) and make yourself familiar with the patient resource
- Create the patient resource making use of the autocomplete functions and the error messages in case the resource is inconsistent

- Now create in the same way a simple observation resource which is validated through the process we followed before
- Status should be "final"

REST OPERATIONS

- [https://wiki.hl7.org/Publicly Available FHIR Servers for testing](https://wiki.hl7.org/Publicly_Available_FHIR_Servers_for_testing)
- Institute eHealth FHIR Server:
 - <http://diam.fh-joanneum.at:8021/hapi-fhir-jpaserver/>
 - <http://diam.fh-joanneum.at:8020/hapi-fhir-jpaserver/>

- POST your own created FHIR resource to the Patient endpoint of the following FHIR server:
 - <http://diam.fh-joaanneum.at:8021/hapi-fhir-jpaserver/>
- Check if you get the "201 Created" status back
- Also check if you got an "id" for your Patient resource

- Get your Patient resource back using the “id” of you patient resource
- Check that you receive the status “200 OK”

- Post your Observation resource to the observation endpoint
- Make sure that the reference is linking to your patient resource
- You should again get back the “201 created” message

- Imagine the Observation you just posted had accidentally a wrong value of an measurement
 - Correct your resource by:
 - Change the status of the observation from “final” to “corrected”
 - Change a value which needs to be corrected
 - Update the resource on the server with the REST PUT command
 - Don’t forget to put the “id” from the original Observation resource on the server in the update resource in the JSON file as well as in the REST request
 - Check if you get the “200 OK” message back
 - Check if the version has changed
 - Check with `_history` the full history of the resource
- *btw PUT you can also use to create a resource if you have the “id” already

BUNDLE TUTORIAL

<https://fhir-drills.github.io/bundle.html>

BUNDLES AND TRANSACTIONS

- Transactions bundles are used to transfer several resource actions to a server
- The server will try to perform all actions (creating resources, updating resources etc.)
- ONLY when all actions of the bundle are successfully performed the transaction is successful (all or nothing)
- In comparison to an collection bundle where the bundle is stored as such

- resourceType = Bundle
- Type=Transaction
- Entry 1:
 - Resource → Patient Resource
- Under the resource comes the "request" which tells the server what to do with the resource
 - "method": "POST"
 - "url": "Patient"
- "fullUrl": "urn:uuid:patient" needs to be added

- Entry 2:
 - Resource → Observation
 - Subject
 - “type”: “Patient”
 - “reference”: “urn:uuid:patient”
- Under the resource comes the “request” which tells the server what to do with the resource (simple POST is enough)

- Post the Bundle again to the server
- Check in the reply if the patient ID has been generated
- Get the observation you just posted back from the server and check if the reference to the patient has been correctly made

- Change the observation in your editor and add an value
- Now we want to update this observation again on the server BUT do not want to create a new patient.
- We can change our Patient resource in the Bundle the following way
 - Instead of “method”: “POST”
 - “method”: “PUT”
 - “url”: “Patient?name=iamtheTestUser”
- What does it do? → instead of creating a new patient it checks if the patients exists and updates the observation otherwise creates the patient
- Check what you get back!

- <https://fhir-drills.github.io/simple-patient.html>

- And:

<https://developer-wp-uks.azurewebsites.net/learn/playing-with-fhir-tut002-creating-and-editing-a-patient/>